

Programação Competitiva

Semana 4: Estratégias de resolução de problemas

Alberto Tavares Duarte Neto

10 de Setembro, 2025

Universidade de Brasília

Problemas de otimização

Definição

Um problema de otimização é um problema no qual devem ser feitas escolhas para maximizar (ou minimizar) uma função.

Problemas de otimização

Definição

Um problema de otimização é um problema no qual devem ser feitas escolhas para maximizar (ou minimizar) uma função.

Exemplo. Uma quantidade n de filmes será exibida no cinema hoje. Cada filme tem início s_i e fim f_i . Qual é a maior quantidade de filmes que uma pessoa pode assistir, considerando que não consegue assistir dois filmes simultaneamente?

Problemas de otimização

Definição

Um problema de otimização é um problema no qual devem ser feitas escolhas para maximizar (ou minimizar) uma função.

Exemplo. Uma quantidade n de filmes será exibida no cinema hoje. Cada filme tem início s_i e fim f_i . Qual é a maior quantidade de filmes que uma pessoa pode assistir, considerando que não consegue assistir dois filmes simultaneamente?

No exemplo, as escolhas são quais filmes assistir, e a função a ser maximizada é a quantidade de filmes assistidos.

Problemas de otimização

Definição

Um problema de otimização é um problema no qual devem ser feitas escolhas para maximizar (ou minimizar) uma função.

Exemplo. Uma quantidade n de filmes será exibida no cinema hoje. Cada filme tem início s_i e fim f_i . Qual é a maior quantidade de filmes que uma pessoa pode assistir, considerando que não consegue assistir dois filmes simultaneamente?

No exemplo, as escolhas são quais filmes assistir, e a função a ser maximizada é a quantidade de filmes assistidos.

Existem várias técnicas para resolver problemas de otimização.

Problemas de otimização

Definição

Um problema de otimização é um problema no qual devem ser feitas escolhas para maximizar (ou minimizar) uma função.

Exemplo. Uma quantidade n de filmes será exibida no cinema hoje. Cada filme tem início s_i e fim f_i . Qual é a maior quantidade de filmes que uma pessoa pode assistir, considerando que não consegue assistir dois filmes simultaneamente?

No exemplo, as escolhas são quais filmes assistir, e a função a ser maximizada é a quantidade de filmes assistidos.

Existem várias técnicas para resolver problemas de otimização.

- Algoritmos gulosos,

Problemas de otimização

Definição

Um problema de otimização é um problema no qual devem ser feitas escolhas para maximizar (ou minimizar) uma função.

Exemplo. Uma quantidade n de filmes será exibida no cinema hoje. Cada filme tem início s_i e fim f_i . Qual é a maior quantidade de filmes que uma pessoa pode assistir, considerando que não consegue assistir dois filmes simultaneamente?

No exemplo, as escolhas são quais filmes assistir, e a função a ser maximizada é a quantidade de filmes assistidos.

Existem várias técnicas para resolver problemas de otimização.

- Algoritmos gulosos,
- Busca completa (e programação dinâmica),

Problemas de otimização

Definição

Um problema de otimização é um problema no qual devem ser feitas escolhas para maximizar (ou minimizar) uma função.

Exemplo. Uma quantidade n de filmes será exibida no cinema hoje. Cada filme tem início s_i e fim f_i . Qual é a maior quantidade de filmes que uma pessoa pode assistir, considerando que não consegue assistir dois filmes simultaneamente?

No exemplo, as escolhas são quais filmes assistir, e a função a ser maximizada é a quantidade de filmes assistidos.

Existem várias técnicas para resolver problemas de otimização.

- Algoritmos gulosos,
- Busca completa (e programação dinâmica),
- Busca binária, no caso em que a função é monotônica,

Problemas de otimização

Definição

Um problema de otimização é um problema no qual devem ser feitas escolhas para maximizar (ou minimizar) uma função.

Exemplo. Uma quantidade n de filmes será exibida no cinema hoje. Cada filme tem início s_i e fim f_i . Qual é a maior quantidade de filmes que uma pessoa pode assistir, considerando que não consegue assistir dois filmes simultaneamente?

No exemplo, as escolhas são quais filmes assistir, e a função a ser maximizada é a quantidade de filmes assistidos.

Existem várias técnicas para resolver problemas de otimização.

- Algoritmos gulosos,
- Busca completa (e programação dinâmica),
- Busca binária, no caso em que a função é monotônica,
- Outras técnicas, que não serão discutidas aqui.

Busca completa

Vamos comentar três exemplos de busca completa:

Vamos comentar três exemplos de busca completa:

- Gerar todos os subconjuntos de um conjunto,

Vamos comentar três exemplos de busca completa:

- Gerar todos os subconjuntos de um conjunto,
- Gerar todas as permutações de n elementos,

Vamos comentar três exemplos de busca completa:

- Gerar todos os subconjuntos de um conjunto,
- Gerar todas as permutações de n elementos,
- Backtracking.

Vamos comentar três exemplos de busca completa:

- Gerar todos os subconjuntos de um conjunto,
- Gerar todas as permutações de n elementos,
- Backtracking.

Problema

Dado um conjunto de $n \leq 20$ inteiros, calcule a maior soma de um subconjunto cuja soma seja $\leq X$ para um inteiro X dado.

Problema

Dado um conjunto de $n \leq 20$ inteiros, calcule a maior soma de um subconjunto cuja soma seja $\leq X$ para um inteiro X dado.

Exemplo. O conjunto $\{5, 7, 11\}$ tem $2^3 = 8$ subconjuntos:

Problema

Dado um conjunto de $n \leq 20$ inteiros, calcule a maior soma de um subconjunto cuja soma seja $\leq X$ para um inteiro X dado.

Exemplo. O conjunto $\{5, 7, 11\}$ tem $2^3 = 8$ subconjuntos:

- $\{\}$,

Problema

Dado um conjunto de $n \leq 20$ inteiros, calcule a maior soma de um subconjunto cuja soma seja $\leq X$ para um inteiro X dado.

Exemplo. O conjunto $\{5, 7, 11\}$ tem $2^3 = 8$ subconjuntos:

- $\{\}$,
- $\{5\}$, $\{7\}$, $\{11\}$,

Problema

Dado um conjunto de $n \leq 20$ inteiros, calcule a maior soma de um subconjunto cuja soma seja $\leq X$ para um inteiro X dado.

Exemplo. O conjunto $\{5, 7, 11\}$ tem $2^3 = 8$ subconjuntos:

- $\{\}$,
- $\{5\}$, $\{7\}$, $\{11\}$,
- $\{5, 7\}$, $\{5, 11\}$, $\{7, 11\}$,

Problema

Dado um conjunto de $n \leq 20$ inteiros, calcule a maior soma de um subconjunto cuja soma seja $\leq X$ para um inteiro X dado.

Exemplo. O conjunto $\{5, 7, 11\}$ tem $2^3 = 8$ subconjuntos:

- $\{\}$,
- $\{5\}$, $\{7\}$, $\{11\}$,
- $\{5, 7\}$, $\{5, 11\}$, $\{7, 11\}$,
- $\{5, 7, 11\}$.

Problema

Dado um conjunto de $n \leq 20$ inteiros, calcule a maior soma de um subconjunto cuja soma seja $\leq X$ para um inteiro X dado.

Exemplo. O conjunto $\{5, 7, 11\}$ tem $2^3 = 8$ subconjuntos:

- $\{\}$,
- $\{5\}$, $\{7\}$, $\{11\}$,
- $\{5, 7\}$, $\{5, 11\}$, $\{7, 11\}$,
- $\{5, 7, 11\}$.

Se $X = 13$, a resposta é 12.

Problema

Dado um conjunto de $n \leq 20$ inteiros, calcule a maior soma de um subconjunto cuja soma seja $\leq X$ para um inteiro X dado.

Exemplo. O conjunto $\{5, 7, 11\}$ tem $2^3 = 8$ subconjuntos:

- $\{\}$,
- $\{5\}$, $\{7\}$, $\{11\}$,
- $\{5, 7\}$, $\{5, 11\}$, $\{7, 11\}$,
- $\{5, 7, 11\}$.

Se $X = 13$, a resposta é 12.

Solução recursiva

```
int solve(int i, int soma) {  
    if(i >= n) {  
        if(soma <= X) return soma;  
        return 0;  
    }  
    return max(solve(i+1, soma), solve(i+1, soma+v[i]));  
}
```

Subconjuntos

Seja b um número inteiro e consideremos sua representação binária. Este número representará um subconjunto S da seguinte forma: o i -ésimo elemento do conjunto original estará no subconjunto escolhido se, e somente se, o i -ésimo bit de b está ligado.

Subconjuntos

Seja b um número inteiro e consideremos sua representação binária. Este número representará um subconjunto S da seguinte forma: o i -ésimo elemento do conjunto original estará no subconjunto escolhido se, e somente se, o i -ésimo bit de b está ligado.

Exemplo. O número 110_2 representa o subconjunto $\{7, 11\}$ de $\{5, 7, 11\}$.

Subconjuntos

Seja b um número inteiro e consideremos sua representação binária. Este número representará um subconjunto S da seguinte forma: o i -ésimo elemento do conjunto original estará no subconjunto escolhido se, e somente se, o i -ésimo bit de b está ligado.

Exemplo. O número 110_2 representa o subconjunto $\{7, 11\}$ de $\{5, 7, 11\}$.

Portanto, iterando por todos os números de 0 até $2^n - 1$ e considerando os subconjuntos associados, estaremos iterando por todos os subconjuntos de um conjunto de n elementos.

Subconjuntos

Seja b um número inteiro e consideremos sua representação binária. Este número representará um subconjunto S da seguinte forma: o i -ésimo elemento do conjunto original estará no subconjunto escolhido se, e somente se, o i -ésimo bit de b está ligado.

Exemplo. O número 110_2 representa o subconjunto $\{7, 11\}$ de $\{5, 7, 11\}$.

Portanto, iterando por todos os números de 0 até $2^n - 1$ e considerando os subconjuntos associados, estaremos iterando por todos os subconjuntos de um conjunto de n elementos.

Solução iterativa

```
for(int b = 0; b < (1 << n); b++) { // (1 << n) = 2^n
    int soma = 0;
    for(int i = 0; i < n; i++) {
        if( (1 << i) & b > 0 ) { // verificando se o i-esimo bit de
            soma += v[i];         // b está ligado
        }
    }
    if(soma <= x) resposta = max(resposta, soma);
}
```

Problema

Dado um baralho com cartas numeradas de 1 até $n \leq 10$, encontre todas as ordens possíveis para o baralho.

Permutações

Problema

Dado um baralho com cartas numeradas de 1 até $n \leq 10$, encontre todas as ordens possíveis para o baralho.

É possível fazer uma solução recursiva. Apresentamos aqui uma solução utilizando a função **next permutation** do c++.

Permutações

Problema

Dado um baralho com cartas numeradas de 1 até $n \leq 10$, encontre todas as ordens possíveis para o baralho.

É possível fazer uma solução recursiva. Apresentamos aqui uma solução utilizando a função **next permutation** do c++.

Solução iterativa

```
vector<int> v(n);  
for(int i = 0; i < n; i++) v[i] = i+1;  
do {  
    for(int i = 0; i < n; i++) {  
        cout << v[i] << " ";  
    }  
    cout << endl;  
} while(next_permutation(v.begin(), v.end()));
```

Definição

Cormen define um **algoritmo guloso** como um algoritmo que toma decisões ótimas localmente, com o intuito de obter uma solução global ótima.

Definição

Cormen define um **algoritmo guloso** como um algoritmo que toma decisões ótimas localmente, com o intuito de obter uma solução global ótima.

Dificuldades:

Definição

Cormen define um **algoritmo guloso** como um algoritmo que toma decisões ótimas localmente, com o intuito de obter uma solução global ótima.

Dificuldades:

- É necessário entender bem as propriedades do problema.

Definição

Cormen define um **algoritmo guloso** como um algoritmo que toma decisões ótimas localmente, com o intuito de obter uma solução global ótima.

Dificuldades:

- É necessário entender bem as propriedades do problema.
- Pode-se perder muito tempo em uma solução gulosa incorreta.

Definição

Cormen define um **algoritmo guloso** como um algoritmo que toma decisões ótimas localmente, com o intuito de obter uma solução global ótima.

Dificuldades:

- É necessário entender bem as propriedades do problema.
- Pode-se perder muito tempo em uma solução gulosa incorreta.
- Em muitos casos, não existe solução gulosa conhecida para o problema.

Definição

Cormen define um **algoritmo guloso** como um algoritmo que toma decisões ótimas localmente, com o intuito de obter uma solução global ótima.

Dificuldades:

- É necessário entender bem as propriedades do problema.
- Pode-se perder muito tempo em uma solução gulosa incorreta.
- Em muitos casos, não existe solução gulosa conhecida para o problema.

Vamos aprender a argumentar a corretude de um algoritmo guloso.

Estudando algoritmos

Problema (B da lista de gulosos)

Samuel quer aprender novos algoritmos. Ele tem o total de X minutos livres. Existem N algoritmos, e cada um deles requer a_i minutos. Encontre a maior quantidade de algoritmos que Samuel pode aprender.

Estudando algoritmos

Problema (B da lista de gulosos)

Samuel quer aprender novos algoritmos. Ele tem o total de X minutos livres. Existem N algoritmos, e cada um deles requer a_i minutos. Encontre a maior quantidade de algoritmos que Samuel pode aprender.

Intuitivamente, queremos estudar primeiro os algoritmos com o menor tempo de aprendizagem primeiro.

Estudando algoritmos

Problema (B da lista de gulosos)

Samuel quer aprender novos algoritmos. Ele tem o total de X minutos livres. Existem N algoritmos, e cada um deles requer a_i minutos. Encontre a maior quantidade de algoritmos que Samuel pode aprender.

Intuitivamente, queremos estudar primeiro os algoritmos com o menor tempo de aprendizagem primeiro.

Solução. Ordene o array de algoritmos, e aprenda os algoritmos na ordem até não houver mais tempo livre.

Estudando algoritmos

Problema (B da lista de gulosos)

Samuel quer aprender novos algoritmos. Ele tem o total de X minutos livres. Existem N algoritmos, e cada um deles requer a_i minutos. Encontre a maior quantidade de algoritmos que Samuel pode aprender.

Prova da corretude. Seja S o subconjunto de algoritmos que serão estudados. Suponha que S seja uma resposta válida (isto é, tenha a maior quantidade possível de algoritmos).

Consideremos o seguinte caso: suponhamos que $i \in S$, $j \notin S$ e que $a_i > a_j$. Ou seja, há um algoritmo com menor tempo do que um que foi escolhido. Então o tempo necessário para aprender todos os algoritmos no conjunto $S \cup \{j\} \setminus \{i\}$ é menor do que o tempo para os de S .

Conclusão. Repetindo o processo acima algumas vezes, provamos que existe um conjunto S que é uma resposta válida tal que: não existem $i \in S$ e $j \notin S$ tal que $a_i > a_j$.

Estudando algoritmos

Problema (B da lista de gulosos)

Samuel quer aprender novos algoritmos. Ele tem o total de X minutos livres. Existem N algoritmos, e cada um deles requer a_i minutos. Encontre a maior quantidade de algoritmos que Samuel pode aprender.

Prova da corretude. Seja S o subconjunto de algoritmos que serão estudados. Suponha que S seja uma resposta válida (isto é, tenha a maior quantidade possível de algoritmos).

Consideremos o seguinte caso: suponhamos que $i \in S, j \notin S$ e que $a_i > a_j$. Ou seja, há um algoritmo com menor tempo do que um que foi escolhido. Então o tempo necessário para aprender todos os algoritmos no conjunto $S \cup \{j\} \setminus \{i\}$ é menor do que o tempo para os de S .

Conclusão. Repetindo o processo acima algumas vezes, provamos que existe um conjunto S que é uma resposta válida tal que: não existem $i \in S$ e $j \notin S$ tal que $a_i > a_j$. Note que **NÃO** precisamos repetir o processo acima.

Método do argumento de troca

O argumento de troca é muito utilizado para mostrar que certas escolhas não precisam ser consideradas.

Método do argumento de troca

O argumento de troca é muito utilizado para mostrar que certas escolhas não precisam ser consideradas.

- **Passo 1.** Consideramos uma possível resposta.

Método do argumento de troca

O argumento de troca é muito utilizado para mostrar que certas escolhas não precisam ser consideradas.

- **Passo 1.** Consideramos uma possível resposta.
- **Passo 2.** Mostramos que, a partir de uma certa troca de escolhas, a nova escolha continua sendo uma resposta.

Método do argumento de troca

O argumento de troca é muito utilizado para mostrar que certas escolhas não precisam ser consideradas.

- **Passo 1.** Consideramos uma possível resposta.
- **Passo 2.** Mostramos que, a partir de uma certa troca de escolhas, a nova escolha continua sendo uma resposta.
- **Passo 3.** Analisamos quais escolhas nunca valem a pena ser consideradas.

Método do argumento de troca

O argumento de troca é muito utilizado para mostrar que certas escolhas não precisam ser consideradas.

- **Passo 1.** Consideramos uma possível resposta.
- **Passo 2.** Mostramos que, a partir de uma certa troca de escolhas, a nova escolha continua sendo uma resposta.
- **Passo 3.** Analisamos quais escolhas nunca valem a pena ser consideradas.

Analisemos novamente a prova do slide anterior:

Método do argumento de troca

O argumento de troca é muito utilizado para mostrar que certas escolhas não precisam ser consideradas.

- **Passo 1.** Consideramos uma possível resposta.
- **Passo 2.** Mostramos que, a partir de uma certa troca de escolhas, a nova escolha continua sendo uma resposta.
- **Passo 3.** Analisamos quais escolhas nunca valem a pena ser consideradas.

Analisemos novamente a prova do slide anterior:

- **Passo 1.** Consideramos um subconjunto S com a maior quantidade possível de algoritmos.

Método do argumento de troca

O argumento de troca é muito utilizado para mostrar que certas escolhas não precisam ser consideradas.

- **Passo 1.** Consideramos uma possível resposta.
- **Passo 2.** Mostramos que, a partir de uma certa troca de escolhas, a nova escolha continua sendo uma resposta.
- **Passo 3.** Analisamos quais escolhas nunca valem a pena ser consideradas.

Analisemos novamente a prova do slide anterior:

- Passo 1. Consideramos um subconjunto S com a maior quantidade possível de algoritmos.
- Passo 2. Se trocarmos um algoritmo $i \in S$ por um algoritmo $j \in S$ tal que $a_i > a_j$, o novo conjunto continua sendo uma resposta válida.

Argumento de troca

Método do argumento de troca

O argumento de troca é muito utilizado para mostrar que certas escolhas não precisam ser consideradas.

- **Passo 1.** Consideramos uma possível resposta.
- **Passo 2.** Mostramos que, a partir de uma certa troca de escolhas, a nova escolha continua sendo uma resposta.
- **Passo 3.** Analisamos quais escolhas nunca valem a pena ser consideradas.

Analisemos novamente a prova do slide anterior:

- Passo 1. Consideramos um subconjunto S com a maior quantidade possível de algoritmos.
- Passo 2. Se trocarmos um algoritmo $i \in S$ por um algoritmo $j \in S$ tal que $a_i > a_j$, o novo conjunto continua sendo uma resposta válida.
- Passo 3. Concluimos que não vale a pena considerar os subconjuntos que não consistem nos menores elementos, já que sempre podemos trocar um algoritmo em S por outro com tempo menor fora de S .

Problema (D da lista de gulosos)

Você tem $n \leq 10^5$ tarefas. Cada tarefa tem uma data de entrega d e uma duração a . A recompensa de terminar uma tarefa é $d - f$, onde f é o tempo de finalização da tarefa. Qual é a maior recompensa, se escolher a ordem de execução das tarefas de forma ótima?

Tarefas e prazos

Problema (D da lista de gulosos)

Você tem $n \leq 10^5$ tarefas. Cada tarefa tem uma data de entrega d e uma duração a . A recompensa de terminar uma tarefa é $d - f$, onde f é o tempo de finalização da tarefa. Qual é a maior recompensa, se escolher a ordem de execução das tarefas de forma ótima?

Ideias?

Tarefas e prazos

Problema (D da lista de gulosos)

Você tem $n \leq 10^5$ tarefas. Cada tarefa tem uma data de entrega d e uma duração a . A recompensa de terminar uma tarefa é $d - f$, onde f é o tempo de finalização da tarefa. Qual é a maior recompensa, se escolher a ordem de execução das tarefas de forma ótima?

Ideias?

Fazer os problemas com o menor prazo primeiro? Consideremos $a_1 = 1000, f_1 = 1, a_2 = 1, f_2 = 10$. O custo de fazer a tarefa 1 primeiro é muito maior do que fazer a tarefa 2 primeiro (faça as contas!).

Problema (D da lista de gulosos)

Você tem $n \leq 10^5$ tarefas. Cada tarefa tem uma data de entrega d e uma duração a . A recompensa de terminar uma tarefa é $d - f$, onde f é o tempo de finalização da tarefa. Qual é a maior recompensa, se escolher a ordem de execução das tarefas de forma ótima?

Ideias?

Fazer os problemas com o menor prazo primeiro? Consideremos $a_1 = 1000, f_1 = 1, a_2 = 1, f_2 = 10$. O custo de fazer a tarefa 1 primeiro é muito maior do que fazer a tarefa 2 primeiro (faça as contas!).

Solução. Fazer os problemas com menor duração primeiro.

Problema (D da lista de gulosos)

Você tem $n \leq 10^5$ tarefas. Cada tarefa tem uma data de entrega d e uma duração a . A recompensa de terminar uma tarefa é $d - f$, onde f é o tempo de finalização da tarefa. Qual é a maior recompensa, se escolher a ordem de execução das tarefas de forma ótima?

Ideias?

Fazer os problemas com o menor prazo primeiro? Consideremos $a_1 = 1000, f_1 = 1, a_2 = 1, f_2 = 10$. O custo de fazer a tarefa 1 primeiro é muito maior do que fazer a tarefa 2 primeiro (faça as contas!).

Solução. Fazer os problemas com menor duração primeiro.

Prova. Consideremos que fazemos a tarefa i e, logo após, a tarefa j , e que $a_i > a_j$, e suponhamos que a recompensa total é C . Se trocarmos i e j na ordem de tarefas executadas, a nova recompensa será

Tarefas e prazos

Problema (D da lista de gulosos)

Você tem $n \leq 10^5$ tarefas. Cada tarefa tem uma data de entrega d e uma duração a . A recompensa de terminar uma tarefa é $d - f$, onde f é o tempo de finalização da tarefa. Qual é a maior recompensa, se escolher a ordem de execução das tarefas de forma ótima?

Ideias?

Fazer os problemas com o menor prazo primeiro? Consideremos $a_1 = 1000, f_1 = 1, a_2 = 1, f_2 = 10$. O custo de fazer a tarefa 1 primeiro é muito maior do que fazer a tarefa 2 primeiro (faça as contas!).

Solução. Fazer os problemas com menor duração primeiro.

Prova. Consideremos que fazemos a tarefa i e, logo após, a tarefa j , e que $a_i > a_j$, e suponhamos que a recompensa total é C . Se trocarmos i e j na ordem de tarefas executadas, a nova recompensa será

$$C - a_i + a_j < C .$$

Problema (H da lista de gulosos)

Existem $n \leq 2 \cdot 10^5$ gravetos, cada um com comprimento $c_i \leq 10^9$. Você pode realizar a seguinte operação quantas vezes quiser: escolher um graveto i e aumentar ou diminuir seu comprimento por $x > 0$. O custo dessa operação é x . Qual é o menor custo para que todos os gravetos tenham o mesmo tamanho?

Problema (H da lista de gulosos)

Existem $n \leq 2 \cdot 10^5$ gravetos, cada um com comprimento $c_i \leq 10^9$. Você pode realizar a seguinte operação quantas vezes quiser: escolher um graveto i e aumentar ou diminuir seu comprimento por $x > 0$. O custo dessa operação é x . Qual é o menor custo para que todos os gravetos tenham o mesmo tamanho?

Solução busca completa. Para todo comprimento d de 1 até 10^9 , calcule o custo de transformar todos os gravetos para o tamanho d . A resposta será o menor de todos os custos.

Problema (H da lista de gulosos)

Existem $n \leq 2 \cdot 10^5$ gravetos, cada um com comprimento $c_i \leq 10^9$. Você pode realizar a seguinte operação quantas vezes quiser: escolher um graveto i e aumentar ou diminuir seu comprimento por $x > 0$. O custo dessa operação é x . Qual é o menor custo para que todos os gravetos tenham o mesmo tamanho?

Solução busca completa. Para todo comprimento d de 1 até 10^9 , calcule o custo de transformar todos os gravetos para o tamanho d . A resposta será o menor de todos os custos. TLE.

Problema (H da lista de gulosos)

Existem $n \leq 2 \cdot 10^5$ gravetos, cada um com comprimento $c_i \leq 10^9$. Você pode realizar a seguinte operação quantas vezes quiser: escolher um graveto i e aumentar ou diminuir seu comprimento por $x > 0$. O custo dessa operação é x . Qual é o menor custo para que todos os gravetos tenham o mesmo tamanho?

Solução busca completa. Para todo comprimento d de 1 até 10^9 , calcule o custo de transformar todos os gravetos para o tamanho d . A resposta será o menor de todos os custos. TLE.

Suponhamos que o custo para transformar todos os gravetos em d seja C . Se considerarmos $d + 1$, o custo C aumenta em 1 para cada graveto com comprimento original $c_i \leq d$, e diminui em 1 para cada graveto com comprimento original $c_j > d$.

Problema (H da lista de gulosos)

Existem $n \leq 2 \cdot 10^5$ gravetos, cada um com comprimento $c_i \leq 10^9$. Você pode realizar a seguinte operação quantas vezes quiser: escolher um graveto i e aumentar ou diminuir seu comprimento por $x > 0$. O custo dessa operação é x . Qual é o menor custo para que todos os gravetos tenham o mesmo tamanho?

Solução busca completa. Para todo comprimento d de 1 até 10^9 , calcule o custo de transformar todos os gravetos para o tamanho d . A resposta será o menor de todos os custos. TLE.

Suponhamos que o custo para transformar todos os gravetos em d seja C . Se considerarmos $d + 1$, o custo C aumenta em 1 para cada graveto com comprimento original $c_i \leq d$, e diminui em 1 para cada graveto com comprimento original $c_j > d$.

Solução gulosa. Transforme todos os gravetos para o comprimento mediano dos gravetos.

Problema (H da lista de gulosos)

Existem $n \leq 2 \cdot 10^5$ gravetos, cada um com comprimento $c_i \leq 10^9$. Você pode realizar a seguinte operação quantas vezes quiser: escolher um graveto i e aumentar ou diminuir seu comprimento por $x > 0$. O custo dessa operação é x . Qual é o menor custo para que todos os gravetos tenham o mesmo tamanho?

Solução busca completa. Para todo comprimento d de 1 até 10^9 , calcule o custo de transformar todos os gravetos para o tamanho d . A resposta será o menor de todos os custos. TLE.

Suponhamos que o custo para transformar todos os gravetos em d seja C . Se considerarmos $d + 1$, o custo C aumenta em 1 para cada graveto com comprimento original $c_i \leq d$, e diminui em 1 para cada graveto com comprimento original $c_j > d$.

Solução gulosa. Transforme todos os gravetos para o comprimento mediano dos gravetos.